
Estrutura de Dados

Aula 04 - Estruturas Lineares: Pilhas e Filas

Rogério Pereira Junior

Área de Telecomunicações
Instituto Federal de Santa Catarina
campus São José
rogerio.pereira@ifsc.edu.br

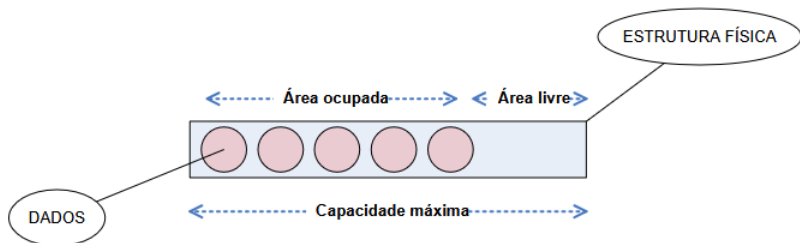
24 de setembro de 2026



Estruturas Lineares



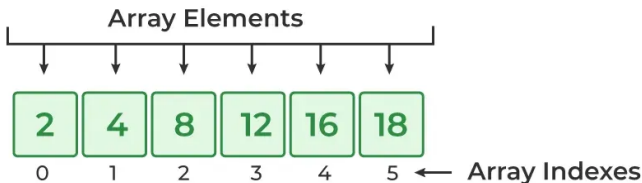
- As estruturas lineares são aquelas em que todos os dados estão acondicionados em uma fileira sequencial única



- Todas as estruturas de dados pressupõem a possibilidade das operações CRUD (create, read, update e delete), mas, dependendo da natureza do problema a ser representado, algumas operações podem ser inibidas e outras podem ser acrescentadas.



- Até o momento, estudamos:
 - array estático;
 - array dinâmico.
- Podemos utilizar essas estruturas para implementar uma **lista de elementos**.

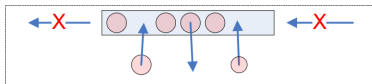


- Em uma lista, podemos realizar operações como:
 - inserção;
 - remoção;
 - consulta;
 - alteração.
- Em geral, podemos acessar ou modificar elementos em diferentes posições da estrutura.

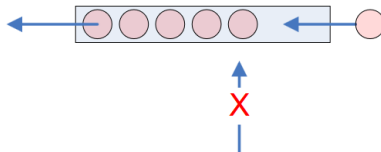


Estruturas Lineares – Restrições de Acesso

- Nem toda estrutura linear permite inserir ou remover elementos em qualquer posição.
- Algumas estruturas impõem **restrições de acesso** para determinar onde as operações podem ocorrer.



Acesso em diferentes posições



Acesso pelas extremidades

Importante

A possibilidade física de realizar uma operação não significa que ela deva ser permitida. As **regras da aplicação** podem restringir as operações disponíveis ao usuário.



Pilha



- Estrutura linear que permite a inserção/extração dos dados apenas por uma das extremidades, chamada **topo**.



- O objetivo do TAD Pilha é representar problemas que envolvam empilhamento de elementos. Ou seja, uma pilha como a conhecemos no mundo real.

- A** você está ocupado com um projeto de longo prazo;
- B** é interrompido por um colega solicitando ajuda em um outro projeto;
- C** enquanto estiver trabalhando em B, alguém da contabilidade aparece para uma reunião sobre despesas de viagem;
- D** durante a reunião, recebe um telefonema de emergência de alguém de vendas e passa alguns minutos resolvendo um problema relacionado a um novo produto.

Quando tiver terminado o telefonema **D**, voltará para a reunião **C**; quando tiver acabado com **C**, voltará para o projeto **B** e, quando tiver terminado com **B**, poderá finalmente voltar para o projeto **A**.

D

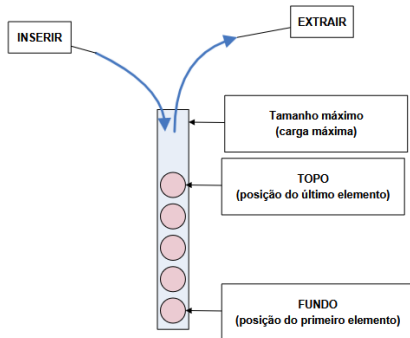
C

B

A



- Elementos são inseridos no **topo**;
- elementos são removidos do **topo**.
- Essa organização segue o princípio **LIFO** (*Last In, First Out*).
 - **Ultimo a entrar será o primeiro a sair**
- Normalmente a inserção e extração são chamados respectivamente de **empilhar** e **desempilhar**
- A retirada ou inserção de um elemento no meio da pilha requer algum tipo de mecanismo que levante os elementos que ficarão acima desse novo para que possa ser encaixado.
- O TAD Pilha convencional não prevê esta operação.



- Inicialmente, vamos implementar a pilha utilizando o **array** estudado nas aulas anteriores.

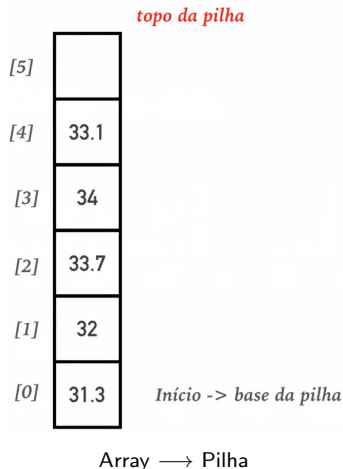


tamanho = 5

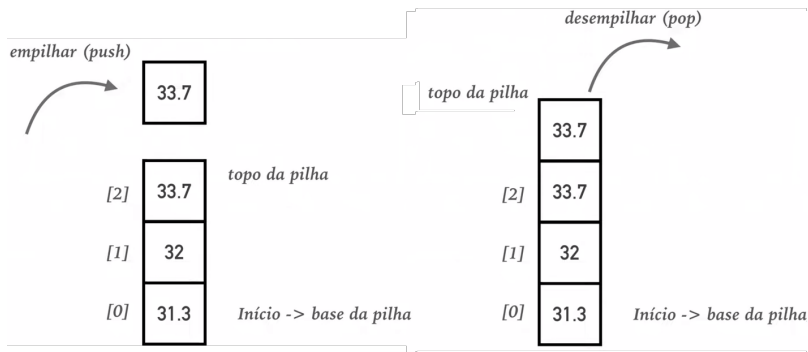
- Neste caso, a posição de inserção e remoção corresponde ao **topo da pilha**.
- Posteriormente, veremos como implementar a pilha utilizando **listas encadeadas**.



- Ao visualizar o array na vertical, temos uma representação natural de uma pilha:
- A pilha é fundamental para o gerenciamento da **execução de processos e funções**.
- É utilizada para armazenar informações de chamadas de funções, como **parâmetros, variáveis locais e endereços de retorno**.
- Cada processo possui uma **pilha de execução** utilizada durante sua execução.



- Computacionalmente falando, precisamos de pelo menos dois métodos no TAD Pilha:



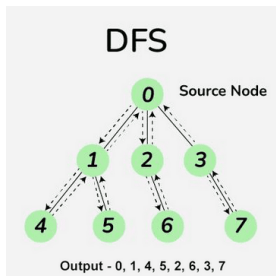
- Para a operação de inserção/empilhar – push(elemento)
- Para a operação de extração/desempilhar – pop()



- A estrutura de dados **pilha** aparece em diversos problemas e algoritmos da Ciência da Computação.
- **Execução de programas**
 - armazenamento de chamadas de funções e variáveis locais;
 - controle do retorno das funções.
- **Compiladores**
 - análise sintática de expressões e programas;
 - verificação de parênteses e outros delimitadores.
- **Algoritmos**
 - busca em profundidade (*DFS*);
 - algoritmos de retrocesso (*backtracking*).
- **Processamento de expressões**
 - conversão entre notações *infixa*, *posfixa* e *prefixa*;
 - avaliação de expressões matemáticas.
- **Aplicações de software**
 - histórico de navegação;
 - operações de *desfazer/refazer*;
 - navegação entre telas e estados.



- Percorre um grafo ou árvore explorando um caminho **o mais profundamente possível**;
- Quando não há mais caminhos a seguir, retorna para o último ponto de decisão;
- Uma **pilha** pode armazenar os vértices que ainda precisam ser explorados.



Funcionamento

- 1 Empilha o vértice inicial;
- 2 Retira o vértice do topo;
- 3 Visita seus vizinhos;
- 4 Empilha os vizinhos ainda não visitados;
- 5 Repete até a pilha ficar vazia.

DFS $\Rightarrow$ Pilha

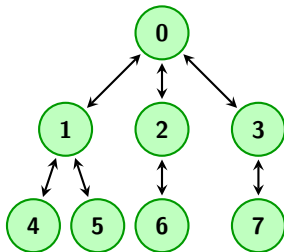
O topo da pilha indica o próximo caminho a ser explorado.



Funcionamento da Pilha (LIFO)

- A DFS iterativa usa uma **Pilha** (*Last In, First Out*).
- Para garantir a ordem de visita da esquerda para a direita ($0 \rightarrow 1 \rightarrow 4 \dots$), empilhamos os vizinhos na **ordem inversa** (direita para a esquerda).
- Em cada passo:
 - 1 Desempilhamos o nó do topo.
 - 2 Registramos o nó na saída (*Output*).
 - 3 Empilhamos seus vizinhos não visitados.

Grafo da Imagem



Passo 1: Inicialização

Ações

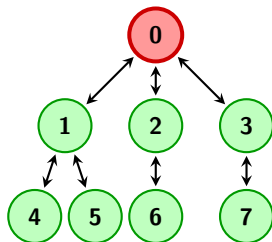
- Inserimos o nó raiz **0** na pilha.
- Marcamos o nó **0** como visitado.

Estado Atual

Pilha (Topo à direita): [0]

Visitados: {0}

Output: []



Passo 2: Processar Nó 0

Ações

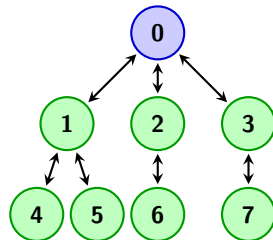
- Desempilhamos o nó **0** → Adicionado ao Output.
- Empilhamos os vizinhos da direita para a esquerda: **3, 2, 1**.

Estado Atual

Pilha (Topo à direita): [3, 2, 1]

Visitados: {0, 1, 2, 3}

Output: [0]



Passo 3: Processar Nó 1

Ações

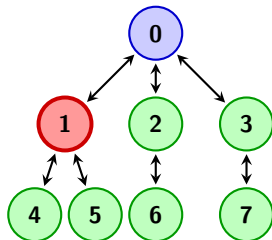
- Desempilhamos o topo (**1**) → Adicionado ao Output.
- Empilhamos os vizinhos de 1 na ordem inversa: **5, 4**.

Estado Atual

Pilha (Topo à direita): [3, 2, 5, 4]

Visitados: {0, 1, 2, 3, 4, 5}

Output: [0, 1]



Passo 4: Processar Nó 4 (Folha)

Ações

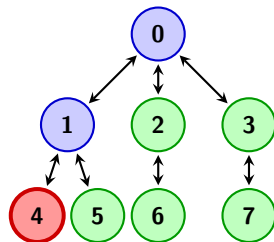
- Desempilhamos o topo (4) → Adicionado ao Output.
- Nó 4 não possui filhos não visitados.

Estado Atual

Pilha (Topo à direita): [3, 2, 5]

Visitados: {0, 1, 2, 3, 4, 5}

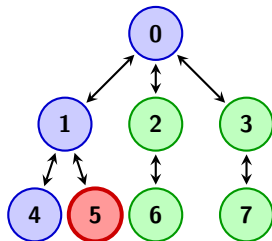
Output: [0, 1, 4]



Passo 5: Processar Nó 5

Ações

- Desempilhamos o topo (**5**) → Adicionado ao Output.
- Sem novos filhos para empilhar.
- Subárvore da esquerda totalmente concluída.



Estado Atual

Pilha (Topo à direita): [3, 2]

Visitados: {0, 1, 2, 3, 4, 5}

Output: [0, 1, 4, 5]



Passo 6: Processar Nó 2 e Nó 6

Ações

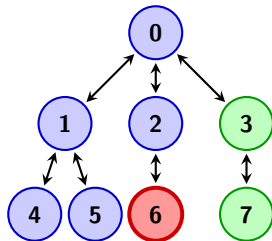
- Desempilhamos 2 → Empilhamos o filho 6.
- Em seguida, desempilhamos 6 → Sem filhos.

Estado Atual

Pilha (Topo à direita): [3]

Visitados: {0, 1, 2, 3, 4, 5, 6}

Output: [0, 1, 4, 5, 2, 6]



Passo 7: Processar Nó 3, Nó 7 e Conclusão

Ações Finais

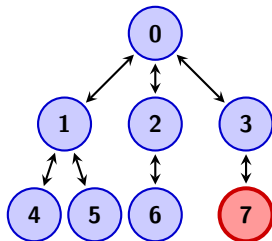
- Desempilhamos 3 → Empilhamos 7.
- Desempilhamos 7 → Pilha fica vazia.
- A busca encerra com sucesso.

Resultado Final

Pilha: [] (Vazia)

Output Final:

0, 1, 4, 5, 2, 6, 3, 7



- Explora possíveis soluções de um problema **passo a passo**;
- A cada etapa, uma escolha é realizada;
- Se a escolha levar a uma situação inválida, ela é **desfeita**;
- O algoritmo retorna ao último ponto de decisão e tenta outra alternativa.



Uso da pilha

- 1 Empilha uma escolha;
- 2 Avança para a próxima decisão;
- 3 Se encontrar uma solução, continua;
- 4 Se encontrar um caminho inválido, **desempilha**;
- 5 Tenta uma nova escolha.

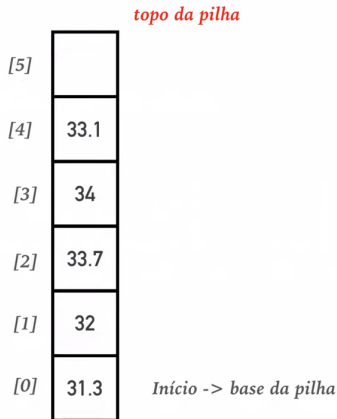
Escolher → Avançar → Desfazer

Exemplos: labirintos, Sudoku, N-Rainhas e geração de combinações.



■ Classe **Pilha**

- Definição
- Verificar quantidade de elementos na pilha
- Empilhar elemento
- Verificar se pilha está vazia
- Espiar/Verificar elemento topo da pilha
- Desempilhar elemento da pilha



- Do zero!
- Via herança
- Via Composição



Prefira Composição a Herança



- Herança cria forte acoplamento
- Composição é mais flexível

Composição > Herança quando o objetivo é reutilizar comportamento



Herança: relação do tipo "é-um"

```
class MotorGasolina extends Motor {  
    ... // herda atributos e métodos de motor  
}
```

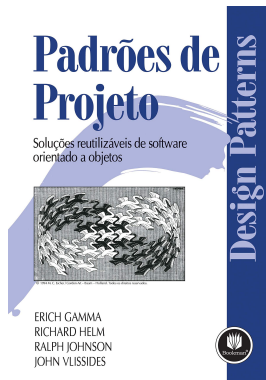


Composição: relação do tipo "possui"

```
class Paine1 {  
    ContaGiros cg;  
    ...  
}
```



- "Herança expõe para subclasses detalhes de implementação das classes pai. Logo, frequentemente diz-se que herança viola o encapsulamento das classes pai. A implementação das subclasses se torna tão acoplada à implementação da classe pai que qualquer mudança nessas últimas pode forçar modificações nas subclasses."



Problema da Herança

- Herança expõe detalhes internos da classe pai
- Subclasses ficam fortemente acopladas à implementação
- Mudanças na classe pai podem quebrar subclasses

Consequência:

- Violação do encapsulamento
- Baixa flexibilidade de manutenção

Diretriz:

- Se houver dúvida entre herança e composição,
- **prefira composição**



Solução via Herança (não recomendada):

```
class Stack extends Vetor {  
    // herda varios metodos  
}
```

Problemas:

- Stack não é um **Vetor** (erro conceitual)
- Herda metodos indevidos
- Quebra a abstracao de pilha

Insight:

- Heranca deve representar relacao "é um"



Composição (recomendado):

```
class Stack {  
    private Vetor elementos;  
  
    Stack(List elementos) {  
        this.elementos = elementos;  
    }  
}
```

Vantagens:

- Mantem encapsulamento
- Evita exposicao de metodos indevidos
- Permite trocar implementacao (ArrayList, LinkedList)

Conclusao:

- Composicao = mais flexivel e desacoplada



Herança ("É Um")

```
126 // Acoplamento Forte
127 class Motor {
128     void ligar() { ... }
129 }
130
131 class Carro extends Motor {
132     void acelerar() {
133         ligar(); // Herda tudo do pai
134     }
135 }
```

- Alta dependência da classe pai.
- Relação estática (compilação).
- Problema da classe base frágil.

Composição ("Tem Um")

```
136 // Baixo Acoplamento
137 interface Motor { void ligar(); }
138
139 class Carro {
140     private Motor motor;
141
142     public Carro(Motor motor) {
143         this.motor = motor; // Injeção
144     }
145 }
```

- Encapsulamento preservado.
- Flexível em tempo de execução.
- Simplifica testes unitários (Mocks).



Exemplo de Uso – Conversão de Decimal para Binário

$$\begin{array}{r} 19 \quad 2 \\ -18 \quad 9 \quad 2 \\ \hline 1 \quad -8 \quad 4 \quad 2 \\ \hline 1 \quad -4 \quad 2 \quad 2 \\ \hline 0 \quad -2 \quad 1 \quad 2 \\ \hline 0 \quad -0 \quad 0 - \text{fim.} \\ \hline 1 \end{array}$$

← $(19)_{10} = (10011)_2$

- Realizamos divisões sucessivas por 2;
- A cada divisão, o **resto** pode ser:

0 ou 1

- Os restos são obtidos na ordem:

$r_0, r_1, r_2, \dots$

- Como utilizamos uma **pilha**, os restos são retirados na ordem inversa.

Decimal → Pilha → Binário



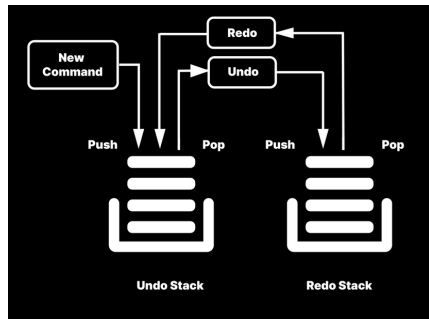
Os elementos da pilha podem ser ocultos do usuário.

- O usuário não deve ter acesso direto aos elementos armazenados na pilha;
- Apenas o elemento do **topo** pode ser removido;
- Sua identidade só é conhecida após a operação `pop()`;
- Isso garante o comportamento definido pela estrutura **LIFO**.

Exemplo: jogo de cartas

As cartas permanecem viradas para baixo. O jogador só conhece a carta quando ela é retirada do topo da pilha.





- Cada novo comando realizado pelo usuário é **empilhado** na pilha de *Undo*;
- Ao executar **Undo**, o último comando realizado é retirado da pilha;
- O comando desfeito é colocado na pilha de **Redo**;
- Ao executar **Redo**, o comando volta a ser aplicado.

LIFO

Última ação realizada
↓
primeira a ser desfeita

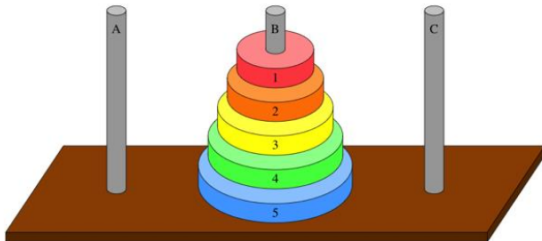


- 1 Um palíndromo é uma palavra, frase ou número pode ser lido de forma igual em ambos os sentidos: da direita para a esquerda e da esquerda para a direita. Ex.: arara, 2002, "anotaram a data da maratona". Crie um programa em JAVA que verifica se uma palavra, frase ou número é um palíndromo.
 - Dicas: Use uma pilha e faça o empilhamento e a verificação a cada desempilhamento com a palavra, frase ou número. Escolha entre palavra, frase ou número, pois as diferenças entre estes pode dificultar na resolução. Se quiser, faça um programa para cada.
- 2 Escreva uma função que receba uma string contendo uma expressão aritmética e retorne se a mesma está com a parentização correta. A função deverá verificar se cada "abre parênteses" tem um "fecha parênteses" correspondente. Ex.:
 - Correto: (()) - (() ()) - () ()
 - Incorreto:)(- (() (-)) ((
- 3 Dada a seguinte sequência de push em uma pilha, crie um programa que elimine o maior e o menor número.
 - 1 pilha.push(30);
 - 2 pilha.push(10);
 - 3 pilha.push(20);
 - 4 pilha.push(50);
 - 5 pilha.push(40);

Dica: Use uma pilha auxiliar.



- No jogo Torre Hanoy o objetivo é mover os discos da haste A para a haste C mantendo a mesma ordem. Em hipótese nenhuma um disco maior poderá ficar sobre um menor. Para que um disco seja movido de A para C, deve-se passar pelo pino B e vice-versa.



- Escreva uma aplicação que implemente este jogo utilizando pilhas. Esta aplicação deve:
 - a) Inicializar os pinos (o primeiro deve conter 3 discos, os demais ficam vazios);
 - b) Criar um método para representar o movimento de um disco de uma haste (pilha) para outra, avaliando se a jogada é legal ou ilegal



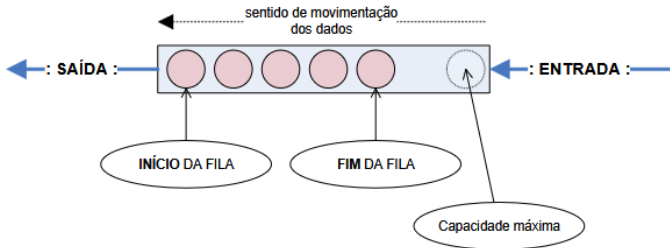
Disponível no sigaa



Fila



- Fila (queue, no inglês) é uma estrutura linear com dois lados de acesso, sendo que a inserção (ou entrada) de dados só é possível por um dos lados e a remoção (ou saída) apenas pelo outro



- O lado de saída é chamado de início ou head (cabeça) e o lado de entrada é o chamado final ou tail (rabo).



- O objetivo do TAD Fila é representar problemas que envolvam enfileiramento de elementos que necessitem de algum tipo de atendimento.



- A fila opera no modelo chamado FIFO (first in first out), ou, traduzindo, o primeiro a entrar será o primeiro a sair.



■ Impressão de documentos

- Documentos aguardam sua vez para serem impressos.

■ Processos em Sistemas Operacionais

- Processos aguardam para utilizar a CPU.

■ Atendimento de requisições

- Servidores processam requisições pendentes.

■ Filas de mensagens

- Sistemas produtores enviam mensagens para consumidores.

■ Redes de computadores

- Pacotes aguardam para serem transmitidos.

■ Simulação de sistemas

- Clientes aguardam atendimento em bancos, hospitais etc.

■ Busca e processamento de dados

- Elementos aguardam para serem processados.

■ Algoritmos

- Busca em largura (*BFS*).

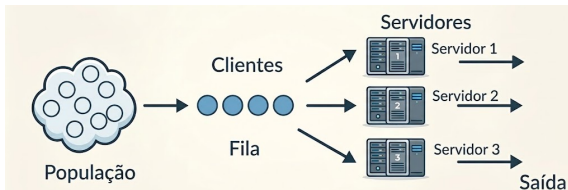
Ideia central

A fila é utilizada quando os elementos precisam ser processados na ordem em que chegaram: **FIFO — First In, First Out**.



Exemplo: servidores Web

Clientes enviam requisições HTTP que precisam ser processadas pelos servidores.



- Várias requisições podem chegar simultaneamente.
- As requisições aguardam em uma **fila de espera**.
- Servidores retiram requisições da fila para processamento.
- Ao liberar um servidor, a próxima requisição pode ser atendida.

Observação

Neste modelo existe uma única fila e diversos servidores para atender os elementos da fila.



O enfileiramento de elementos sempre ocorre quando:

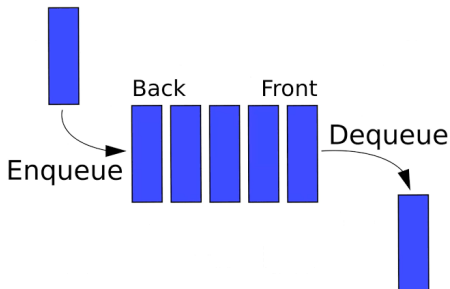
- O consumidor/servidor está indisponível e elementos precisam ser atendidos para passar para outra etapa.
- O consumidor/servidor está disponível, mas a taxa de chegada de elementos por unidade de tempo é maior que a capacidade de atendimento do consumidor/servidor



- A possibilidade de leitura, ou não, dos elementos da fila depende da natureza da aplicação.
- Embora os dois métodos relacionados acima sejam suficientes para a operação da fila, outros três métodos costumam aparecer nas implementações mais básicas:
 - `peek()` – lê o primeiro elemento da fila sem removê-lo.
 - `isFull()` – verifica se o tamanho da fila está na capacidade máxima do contêiner.
 - `isEmpty()` – verifica se a fila está vazia (0 elemento).



Tipicamente precisamos dos seguintes métodos:

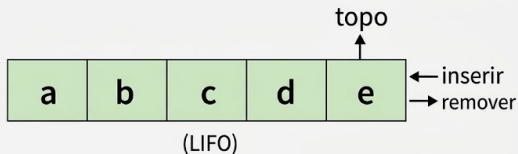


- Para a operação de inserção – `enqueue(elemento)` – (enfileirar)
- Para a operação de extração – `dequeue()` – (desenfileirar)



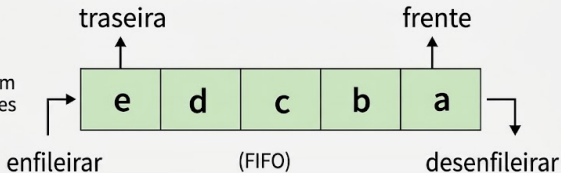
Pilha

Inserção e Deleção ocorrem na mesma extremidade



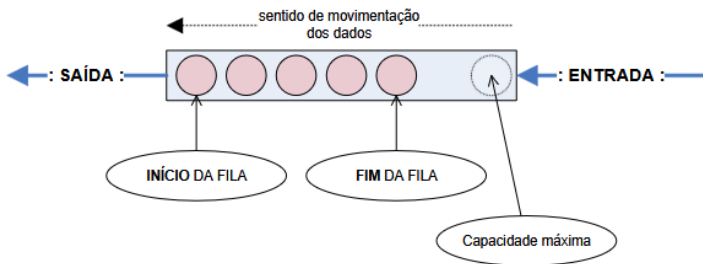
Fila

Inserção e Deleção ocorrem em extremidades diferentes



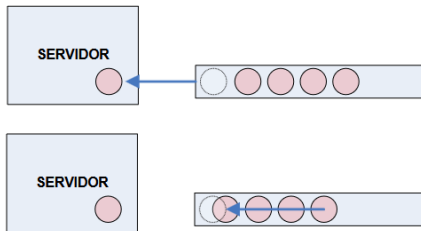
■ Classe Fila

- Enfileirar: Colocar um item no final da fila
- Desenfileirar: Remover um item do início da fila
- Ver início da fila: Mostra o elemento que está no início da fila
- Tamanho da fila
- Verificar se esta vazia



■ Processo de Remoção:

- O elemento na primeira posição (índice 0) é removido da fila.



■ Necessidade de Deslocamento (Shift):

- Para manter o início da fila fixo na posição 0, todos os elementos remanescentes precisam ser deslocados uma posição para a esquerda.
- Se a fila possui n elementos, é necessário mover $n - 1$ elementos a cada remoção.

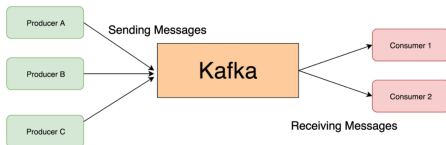
■ Complexidade de Tempo (Notação Big-O):

- Complexidade: $\mathcal{O}(n)$ no pior.



Exemplo: sistemas de mensageria

Sistemas como o **Apache Kafka** permitem que aplicações produzam mensagens que são armazenadas temporariamente até serem consumidas por outras aplicações.



- **Produtores** enviam mensagens para uma fila ou tópico.
- As mensagens ficam **aguardando o consumo**.
- **Consumidores** retiram as mensagens para processamento.
- Vários produtores e consumidores podem trabalhar simultaneamente.

Ideia principal

A fila desacopla quem **produz** a mensagem de quem **processa** a mensagem.





- **2 produtores** geram pacotes de rede;
- Os pacotes são armazenados em uma **única fila**;
- **3 consumidores** retiram os pacotes da fila para processá-los;
- Cada pacote contém informações como:
 - origem;
 - destino;
 - dados da mensagem.



Implementar a Fila a partir da classe **Vetor** via decomposição.



Exercício de fixação 1 - Em Dupla

Implemente um programa de teste para simular o comportamento de uma fila de requisições em um computador servidor de forma que:

- 1 O algoritmo deve executar diversas vezes consecutivas, chamadas ciclos, para simular o tráfego de requisições ao longo do tempo (laço for ou while).
- 2 A fila deve ser acondicionada em um vetor de objetos com dados diversos.
- 3 O servidor tem diversos processadores paralelos.
- 4 A cada ciclo de simulação:
 - O servidor atende tantas requisições quantos forem os seus processadores (tempo de atendimento igual para todas as requisições).
 - Chegam de 0 a N novas requisições na fila.
- 5 O objetivo é poder analisar a probabilidade de perda de requisições (a fila está cheia por ocasião da chegada de uma nova requisição) em diversas situações/configurações:
 - Quantidade máxima de requisições por ciclo (0 a max). do vetor.
 - Quantidade de processadores do servidor.

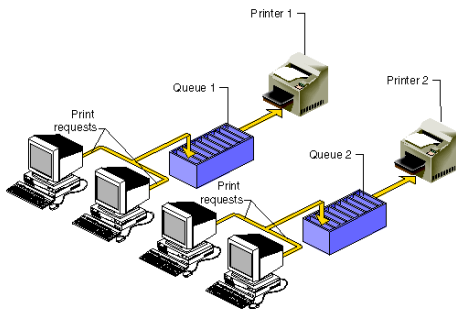
Criatividade fica por sua conta!

Apresentação em mesa das soluções na próxima aula!



Exercício de fixação 2

Em sistemas operacionais, os documentos enviados à impressora ficam numa fila de impressão (spooler). Para garantir eficiência nas entradas e saídas constantes, utiliza-se uma fila circular, que reaproveita o espaço da memória sem deslocar elementos.



Crie um programa de teste que simule:

- Usuários enviando documentos para impressão (ex.: "Relatório.pdf", "Contrato.docx", "Planilha.xlsx").
- A impressora retirando documentos da fila circular e imprimindo-os.
- O reaproveitamento do espaço do array quando o índice fim dá a volta.



- 1 Uma determinada loja que presta serviços relacionados a telefonia celular está sofrendo muitas críticas de seus clientes, os quais perdem muito tempo nas filas para atendimento. Com o objetivo de desenvolver um plano de ação para melhoria do atendimento ao cliente, com a diminuição do tempo de espera nas filas, o responsável pela loja contratou você para desenvolver um estudo sobre as filas. Para isso, solicitou que desenvolvesse uma simulação do atendimento. Na loja existem 4 atendentes, e a cada minuto chegam entre 3 e 11 clientes; cada atendente demora entre 5 e 15 minutos para atender um cliente. Para cada atendente existe uma fila; os clientes escolhem aquela que está vazia ou com menos pessoas. Para cada cliente que entra na fila, deve-se registrar quanto tempo ficou na fila, (diferença entre o horário que chegou na fila e horário de atendimento). O algoritmo ou programa deverá informar o tempo médio que cada cliente permaneceu na fila e o total de atendimentos realizados por atendente.

